

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..20.2

JWT TOKEN MANAGER

 Copy as Markdown

JWT token manager

The `JwtTokenManager` uses JWT (JSON Web Tokens) for stateless authentication. It works with two kinds of tokens. The access token is short-lived and is sent with every request. The refresh token is long-lived and is used only to obtain a new access token when the old one expires. This token manager provides:

- Stateless authentication (no database queries for validation).
- Access tokens with short lifetimes.
- Refresh tokens for long-term authentication.
- Automatic token rotation.

WEB APPS

With [cookie-based web authentication](#) enabled, browsers keep the access token in memory only and receive the refresh token as an `httpOnly` cookie.

Server-side configuration

The `JwtTokenManager` is created by passing a `JwtConfig` object in the `tokenManagerBuilders` list of `pod.initializeAuthServices()`:

```
pod.initializeAuthServices(  
  tokenManagerBuilders: [  
    JwtConfig(  
      // The pepper used to hash the refresh token secret.  
      refreshTokenHashPepper: pod.getPassword(  
        'jwtRefreshTokenHashPepper',  
      )!,  
      // The algorithm used to sign the tokens.  
      algorithm: JwtAlgorithm.hmacSha512(  
        SecretKey(pod.getPassword('jwtHmacSha512PrivateKey')!),  
      ),  
    ),  
  ],  
)
```



```
),  
1,  
);
```

TIP

You can use the `JwtConfigFromPasswords` constructor instead of the `JwtConfig` above. It loads the credentials from the `config/passwords.yaml` file or environment variables. It expects either the following keys in the file:

- `jwtRefreshTokenHashPepper`
- `jwtHmacSha512PrivateKey`

Or the following environment variables:

- `SERVERPOD_PASSWORD_jwtRefreshTokenHashPepper`
- `SERVERPOD_PASSWORD_jwtHmacSha512PrivateKey`

The app calls a refresh endpoint to renew expired access tokens without asking the user to sign in again. Extend the abstract endpoint to expose it on the server:

```
import 'package:serverpod_auth_idp_server/core.dart' as core;  
  
class RefreshJwtTokensEndpoint extends core.RefreshJwtTokensEndpoint {}
```

Finally, run `serverpod generate` so the app can call the endpoint. If your server runs with `serverpod start`, this happens automatically.

Basic configuration options

- `algorithm`: Required. The algorithm to use for signing tokens (HMAC SHA-512, HMAC SHA-256 or ECDSA SHA-512).
- `refreshTokenHashPepper`: Required. A secret pepper for hashing refresh tokens (see [storing secrets](#) for what a pepper is). Must be at least 10 characters long, but [the recommended length is 32 bytes](#).

Token algorithms

There are three supported token algorithms:

- **HMAC SHA-512**: Use HMAC SHA-512 for symmetric key signing.

```
algorithm: JwtAlgorithm.hmacSha512(  
  SecretKey(pod.getPassword('authenticationTokenPrivateKey')!),
```

```
),
```

- **HMAC SHA-256:** Use HMAC SHA-256 for symmetric key signing.

```
algorithm: JwtAlgorithm.hmacSha256(  
    SecretKey(pod.getPassword('authenticationTokenPrivateKey')!),  
),
```

- **ECDSA SHA-512:** Use ECDSA SHA-512 for asymmetric key signing.

```
algorithm: JwtAlgorithm.ecdsaSha512(  
    privateKey: ecPrivateKey,  
    publicKey: ecPublicKey,  
),
```

As of now, the `JwtConfigFromPasswords` only supports HMAC SHA-512. To use HMAC SHA-256 or ECDSA SHA-512, use `JwtConfig` and pass the keys manually.

Extra configuration options

Common configuration options for the `JwtTokenManager`. For more details on all options, check the `JwtConfig` in-code documentation.

```
final jwtConfig = JwtConfigFromPasswords(  
    // Optional: Set fallback algorithms for token verification  
    // This is useful for allowing old tokens to be validated after a rotation.  
    fallbackVerificationAlgorithms: [  
        JwtAlgorithm.hmacSha512(  
            SecretKey(pod.getPassword('fallbackJwtPrivateKey')!),  
        ),  
    ],  
    // Optional: Configure token lifetimes  
    accessTokenLifetime: Duration(minutes: 10),  
    refreshTokenLifetime: Duration(days: 14),  
    // Optional: Add custom claims to tokens.  
    // Claims will be included in every access token and sent with requests.  
    extraClaimsProvider: (session, context) async {  
        return {  
            'userRole': 'admin',  
            'organizationId': 'org-123',  
        };  
    },  
    // Check the [JwtConfig] documentation for more options.  
);
```

Attaching custom metadata to tokens

You can attach custom metadata to each JWT refresh token by providing an `onRefreshTokenCreated` callback. This is useful for storing device information, IP address, user agent, or any other data you need to query or use later (for example, to list or revoke tokens by device). The callback runs when a refresh token is created, within the same transaction as the token insert.

Define a server-only table that relates to `RefreshToken` and store your metadata there. Example schema:

```
class: TokenMetadata
serverOnly: true
table: token_metadata
fields:
  ### The [RefreshToken] this metadata belongs to
  refreshToken: module:serverpod_auth_core:RefreshToken?, relation(onDelete=Cascade)

  ### The name of the token
  name: String?

  ### Device information for the token
  deviceName: String?

  ### IP address from which the token was created
  ipAddress: String?

  ### User agent string
  userAgent: String?

indexes:
  refresh_token_id_unique_idx:
    fields: refreshTokenId
    unique: true
```

Then configure the callback in your JWT config:

```
JwtConfigFromPasswords(
  onRefreshTokenCreated:
    (
      final session, {
        required final authUserId,
        required final refreshTokenId,
        required final transaction,
      }) async {
      await TokenMetadata.db.insertRow(
        session,
        TokenMetadata(
          refreshTokenId: refreshTokenId,
          name: 'general-token',
          ipAddress: session.request?.connectionInfo.remote.address.toString(),
          userAgent: session.request?.headers.userAgent,
        ),
        transaction: transaction,
      );
    },
),
```

```
),
```

To revoke tokens based on your custom metadata, query the metadata table for the token IDs you want to revoke and call `revokeToken` for each:

```
final tokenMetadata = await TokenMetadata.db.find(
  session,
  where: (final row) => row.deviceName.equals('Old Device'),
);

for (final row in tokenMetadata) {
  await AuthServices.instance.tokenManager.revokeToken(
    session,
    tokenId: row.refreshTokenId.toString(),
  );
}
```

Attaching metadata when creating tokens from an endpoint

The `onRefreshTokenCreated` callback is global and runs for every new refresh token, including those created by identity providers. An endpoint that creates a personal access token (PAT) or CLI token often has its own parameters, such as a token name or label, which the callback can't see.

In that case, create the token with `AuthServices.instance.tokenManager.createToken` rather than `issueToken`, which is for [sign-in flows](#). Then use the returned `AuthSuccess.jwtRefreshTokenId` to insert your metadata with the endpoint's parameters:

```
final authSuccess = await AuthServices.instance.tokenManager.createToken(
  session,
  authUserId: userId,
  method: 'pat',
  scopes: {Scope.admin},
);

await TokenMetadata.db.insertRow(
  session,
  TokenMetadata(
    refreshTokenId: authSuccess.jwtRefreshTokenId,
    name: tokenName, // from your endpoint parameter
    deviceName: deviceName, // from your endpoint parameter
    ipAddress: session.request?.connectionInfo.remote.address.toString(),
    userAgent: session.request?.headers.userAgent,
  ),
);
```

Client-side configuration

The `JwtTokenManager` needs no extra configuration in your app. The client includes the access token in requests automatically and refreshes it when it is 30 seconds from expiring. If the refresh token itself expires, the client signs the user out the next time it validates the tokens. Your app decides what to show next, for example by listening to [authentication state changes](#).

Related

- [Managing tokens](#): issue, validate, revoke, and list tokens.
- [Server-side sessions token manager](#): the database-backed alternative.
- [Setup](#): where token managers are configured.